2º DAW · CURSO 2026/2027

PROYECTO INTEGRADO
COFFEESTATION

Documento unificado · DWEC + DIW + DWES

Cómo está organizado

Se conserva la información común de los tres enunciados y, cuando aparecen los casos específicos, se selecciona únicamente CoffeeStation. Se excluyen las fichas particulares de FitZone, CineTown y ParkControl, salvo menciones necesarias para explicar el ecosistema o los contratos de integración.

SECCIONES Y CAMBIOS DE DOCUMENTO MARCADOS

Orden: Desarrollo Web en Entorno Cliente · Diseño de Interfaces · Desarrollo Web en Entorno Servidor

DOCUMENTO 1 DE 3

DWEC · Entorno Cliente

Requisitos comunes de Angular, programación en cliente, comunicación asíncrona, seguridad, evaluación y entrega. En las páginas de casos se mantiene CoffeeStation.

A continuación se incluyen las páginas seleccionadas del documento original.

PROYECTO INTEGRADO · RA4 Y RA7

Proyecto Integrado: SafaReyes Web

Aplicación cliente en Angular para el negocio del ecosistema SafaReyes

Entrega	Modalidad	Tecnología
Frontend del proyecto	**Individual**	**Angular · TypeScript**

Partiendo del enunciado del proyecto **Backend con Spring Boot · Ecosistema SafaReyes** de Desarrollo Web en Entorno Servidor, a continuación se detalla lo que se solicitará por parte de la asignatura de DWEC: la **aplicación web en Angular** del mismo caso que tienes asignado en DWES (CoffeeStation, FitZone, CineTown o ParkControl), que será la que usen los clientes, el personal y la administración de ese negocio.

IMPORTANTE — En DWEC se evalúa el código cliente. Si tu API de DWES no estuviese terminada, la aplicación puede funcionar con datos simulados (un servidor de prueba en Express como el visto en apuntes de clase, o json-server), siempre que respete exactamente las rutas y los campos JSON del enunciado de DWES. Indica en la memoria qué partes funcionan contra la API real y cuáles contra datos simulados.

1. Estructura base en Angular

Deberá ser un proyecto **standalone** creado con Angular CLI, con todos los componentes necesarios para completar las funcionalidades de tu caso (por ejemplo: carta, detalle-producto, carrito, mis-pedidos, login, panel de administración…).

- **Modelos TypeScript** para las entidades de tu caso: interfaces que reflejen el JSON que envía y recibe la API (con los mismos nombres de campo que en DWES) y clases con comportamiento para la lógica de la aplicación (por ejemplo `Producto`, `Carrito`, `Pedido`, `Socio`, `Sesion`, `Ticket`…).
- **Servicios** para manejar la lógica y la comunicación con la API (por ejemplo `ProductoService`, `PedidoService`, `AuthService`). Los componentes no hacen peticiones HTTP directamente: se las piden a un servicio.
- **Rutas** para cada vista de la aplicación y **componentes reutilizables** que se comuniquen mediante `input()` / `output()`.
- Proyecto **organizado en carpetas** por responsabilidad. Se propone esta estructura, que puedes adaptar justificándolo en la memoria:

ESTRUCTURA DEL PROYECTO

```
src/app
├── models          → interfaces y clases del dominio (producto.ts, pedido.ts…)
├── services        → servicios de datos y de autenticación
├── components      → componentes reutilizables (tarjeta, paginador, filtro…)
├── pages           → una carpeta por vista (carta, detalle, carrito, login, admin…)
├── guards          → protección de rutas por sesión y rol
├── interceptors    → añadir el token a las peticiones
├── utils           → funciones propias (cálculos, validaciones, formateo…)
├── app.routes.ts
└── app.config.ts
src/environments    → URL de la API (desarrollo / producción)
```

2. Lógica de programación cliente (RA4)

Se valorará:

- Uso de **funciones predefinidas** del lenguaje: métodos de arrays (`map`, `filter`, `reduce`, `find`, `some`, `sort`…), de cadenas, de `Math`, `Number` y `Date`, y de `JSON` y el almacenamiento web.
- **Funciones propias** para la lógica de negocio en cliente (calcular totales, aplicar descuentos, validar un DNI o una matrícula, formatear duraciones…), documentadas y reutilizadas.
- **Gestión de arrays** para almacenar y procesar la información: inserción, eliminación, búsqueda, filtrado, ordenación y agrupación (carrito, listados, historiales…).
- **Clases** de dominio con propiedades, métodos, encapsulación y al menos un caso de **herencia y polimorfismo** justificado en tu caso.
- Uso de los **objetos definidos por el usuario** en servicios y componentes (no solo declararlos).
- **Código comentado** y estructurado.

3. Comunicación asíncrona (RA7)

Se valorará:

- Integración con el backend mediante **HttpClient** (GET, POST, PUT, PATCH y DELETE) desde los servicios.
- **Carga dinámica** de los listados y detalles, con paginación y filtros enviados como parámetros de la petición.
- **Formularios** para el alta y la edición de datos, con validación en cliente y mostrando los errores que devuelve el servidor.
- Uso de **Observable** y `subscribe()` para actualizar las vistas sin recargar la página, mostrando al usuario cuándo se está cargando, cuándo hay un error y cuándo la operación ha terminado bien.
- **Inicio de sesión** con el token JWT de la API: guardar la sesión, enviar el token en las peticiones y mostrar a cada rol solo lo que puede hacer.
- Al menos una vista que se **actualice sola** cada cierto tiempo (la pantalla destacada de tu caso).

4. Aplicación final, documentación y exposición

Se valorará el atractivo y la funcionalidad de la aplicación en general, el uso de una librería de componentes (Angular Material o Bootstrap) de forma coherente, posibilidad de carga de imágenes (Cloudinary, Firebase Storage, Supabase Storage, AWS S3…) en la creación de items desde la administración (si aplica a tu caso), el vídeo explicando el trabajo realizado y la documentación entregada.

Como mejora, se valorará incluir un **widget del complejo**: un bloque que muestre en directo datos públicos de la API de un compañero con otro caso (por ejemplo, la cafetería mostrando las sesiones de cine de hoy, o el gimnasio mostrando las plazas libres del parking).

5. Requisitos funcionales

Los requisitos funcionales y las reglas de negocio son los de tu caso en el documento de DWES. Desde el cliente, la aplicación deberá ofrecer como mínimo estas vistas y consumir los 16 endpoints de negocio de tu caso, además del registro y el inicio de sesión:

Caso	Cliente / público	Personal	Administración
CoffeeStation	Carta con filtros, detalle con alérgenos, carrito con cupón, mis pedidos y seguimiento del turno	**Tablero de barra** con los pedidos por estado; disponibilidad de productos y categorías	Gestión de carta y cupones; informes de ventas
FitZone	Planes, actividades del día y reservas; mi ficha y mis cuotas	**Terminal del torno** con aforo; alta y búsqueda de socios; cobro de cuotas y morosos	Activar socios, generar cuotas; informe de ingresos
CineTown	Cartelera con filtros, sesiones por fecha, **compra de entradas**, mis compras y cancelación	Venta en taquilla y consulta de compras	Gestión de películas y programación de sesiones; informes
ParkControl	Disponibilidad por planta y tipo; tarifa vigente; mis tickets (abonado)	Barrera de entrada, mapa de plazas, **cajero** con importe en curso y tickets	Plazas, tarifas y abonados; informes

La vista marcada en negrita es la **pantalla destacada** de cada caso: es la que debe actualizarse sola (requisito del apartado 3).

Requisitos de desarrollo

Categoría	Descripción
Programación en cliente	- Implementación de funciones y métodos en componentes y servicios - Uso de funciones predefinidas del lenguaje (map, filter, reduce, etc.) - Creación de funciones propias para la lógica de negocio en cliente
Estructuras de datos	- Gestión de arrays para almacenar y procesar información (productos, carrito, socios, sesiones, tickets…) - Operaciones sobre colecciones: inserción, eliminación, filtrado, búsqueda, ordenación y agrupación
POO en TypeScript	- Interfaces para los datos de la API y clases para las entidades del dominio - Propiedades, métodos, encapsulación, herencia y polimorfismo - Uso de objetos definidos por el usuario en la lógica de la aplicación
Comunicación asíncrona	- Consumo de los endpoints del backend con `HttpClient` (GET, POST, PUT, PATCH, DELETE) - Uso de Observables (`subscribe`, `pipe`) y gestión de errores en las peticiones - Envío de datos en JSON y como parámetros de la URL (paginación y filtros)
Actualización dinámica	- Refresco de la interfaz al modificar datos (carrito, pedidos, reservas, compras, tickets…) - Uso correcto de binding, `@if` / `@for`, `input()` / `output()` y formularios - Vista con actualización periódica
Seguridad en cliente	- Registro e inicio de sesión contra la API - Envío del token en las peticiones y cierre de sesión al caducar - Rutas y opciones de menú según el rol
Depuración y documentación	- Comentarios claros en el código y organización de archivos - Uso de las herramientas de depuración del navegador / Angular DevTools - Mensajes al usuario ante errores de las peticiones

Requisitos no funcionales

Categoría	Descripción
Eficiencia en cliente	Carga fluida de los listados y operaciones sin recargas completas de la página.
Limpieza de código	Nombres adecuados, modularidad, uso de servicios y separación de responsabilidades.
Accesibilidad y UX	Interfaz clara y adaptable, con indicación visual de carga, error y éxito en las operaciones.
Compatibilidad	Funciona correctamente en Chrome y Firefox como mínimo.

6. Evidencias de aprendizaje asociadas a los CE

CE	Evidencia observable
4.a – 4.b (Funciones)	Funciones predefinidas y funciones propias en servicios, componentes y utilidades para gestionar los datos (totales, descuentos, validaciones, formateo…).
4.c – 4.d (Arrays)	Código donde se gestionan colecciones en cliente: carrito, listados, filtros, agrupaciones, historiales.
4.e – 4.h (POO en cliente)	Archivos `.ts` con interfaces y clases de dominio, herencia y polimorfismo, e instanciación y uso de esos objetos en la aplicación.
4.i (Depuración y documentación)	Comentarios JSDoc/TSDoc, mensajes de consola controlados, manejo de errores y estructura de proyecto clara.
7.a – 7.b (Análisis de la asincronía)	Memoria técnica justificando el uso de HTTP y Observables, sus ventajas frente a la comunicación síncrona y los mecanismos disponibles.
7.c – 7.f (HttpClient y Observables)	Servicios Angular consumiendo la API REST, envío de JSON y parámetros, gestión de la sesión y actualización dinámica de la interfaz.
7.g (Compatibilidad)	Evidencia de pruebas en al menos dos navegadores (capturas en la memoria o en la exposición).
7.h – 7.i (Uso y depuración de librerías)	Uso de HttpClient, RxJS y la librería de componentes, manejo de errores y análisis de las librerías utilizadas.

7. Planificación orientativa

AUTOEVALUACIÓN — Esta tabla no se entrega ni se corrige: sirve para que compruebes si vas bien de tiempo. Solo se evalúa la entrega final. La columna central indica lo que se irá viendo en clase y que puedes aplicar en esa parte del proyecto.

Semana	Al terminar la semana deberías tener…
26 – 30 oct	El proyecto creado, la estructura de carpetas y la lista de vistas y rutas de tu caso
2 – 6 nov	Los modelos (interfaces y clases) y las funciones propias de tu caso
9 – 13 nov	Las vistas principales navegables, con datos de prueba
16 – 20 nov	Los listados y detalles cargando datos de la API (real o simulada)
23 – 27 nov	Los formularios de alta y edición y el flujo principal de tu caso
30 nov – 4 dic	Login, registro y la aplicación adaptada a cada rol
9 – 11 dic	La aplicación conectada a tu API y la pantalla destacada actualizándose sola. Entrega Final

8. Entrega asociada a DWEC

La entrega deberá incluir:

- **Código fuente completo** en un repositorio de GitHub: componentes, servicios, modelos y plantillas, con un README que explique cómo levantar el proyecto (y la API o los datos simulados que necesite).
- **Memoria del proyecto** que incluya:
  - Evidencias de pruebas de comunicación cliente-servidor (pestaña Red del navegador).
  - Capturas o explicación de las herramientas de depuración utilizadas.
  - Pruebas en al menos dos navegadores distintos.
  - Evaluación de ventajas e inconvenientes de usar comunicación asíncrona y de los mecanismos disponibles.
  - Clasificación y análisis de librerías que permitan actualizaciones dinámicas.
  - Justificación de las decisiones técnicas (estructura, clases, dónde se guarda la sesión…).
- **Vídeo** grabado explicando la aplicación funcionando con los distintos roles, explicando las diferentes pantallas. En caso de dudas, en clase se podrá pedir explicar o modificar en directo alguna parte del código.

La nota del proyecto dependerá en un **25 %** de la documentación, presentación, vídeo y aplicación en general, mientras que el **75 %** restante será la nota obtenida en los CE puros.

La nota final de cada CE será la nota obtenida en dicho CE (75 %) más la nota de la documentación, presentación, exposición en vídeo y aplicación en general (25 %), para considerar la forma de presentar y defender los CE.

AUTORÍA — Debes ser capaz de explicar y modificar todo el código que entregas. La parte que no sepas explicar ante una posible defensa no se tendrá en cuenta para los CE correspondientes. Se aplica la normativa del módulo sobre copia, plagio y uso de IA.

DOCUMENTO 2 DE 3

DIW · Diseño de Interfaces

Se incluyen las secciones comunes de UX/UI y los requisitos específicos de CoffeeStation. Se omiten las fichas particulares de los otros negocios.

A continuación se incluyen las páginas seleccionadas del documento original.

PROYECTO INTEGRADO INDIVIDUAL

Diseño de Interfaces Web

Ecosistema SafaReyes

Del problema de un cliente real a una interfaz usable y accesible: investigación, arquitectura de información, prototipado, sistema visual, responsive design, multimedia, accesibilidad y pruebas de usabilidad.

CoffeeStation	FitZone	CineTown	ParkControl
Hostelería	Deporte	Ocio	Movilidad

Ciclo Formativo de Grado Superior · Desarrollo de Aplicaciones Web

COORDINACIÓN

Este documento define exclusivamente los requisitos evaluables en Diseño de Interfaces Web. El producto final se desarrolla de forma coordinada con Desarrollo Web en Entorno Cliente y Desarrollo Web en Entorno Servidor, utilizando el mismo caso y los mismos actores y reglas funcionales.

Curso 2026/2027

Documento de requisitos del proyecto · Diseño de Interfaces Web

Contenido

1 Presentación del proyecto

2 Resultados de aprendizaje y evidencias

3 Requisitos comunes de Diseño de Interfaces

4 Entregables y estructura del repositorio

5 Fases y temporalización

6 Caso 1 · CoffeeStation

7 Caso 2 · FitZone

8 Caso 3 · CineTown

9 Caso 4 · ParkControl

10 Contratos de interfaz con Cliente y Servidor

11 Evaluación y lista de comprobación

12 Referencias profesionales

CÓMO USARLO

Las secciones 1 a 5, 10 y 11 son comunes a todo el grupo. De las secciones 6 a 9 cada alumno debe seguir la de su caso. Las pantallas indicadas son requisitos de interfaz; no implican que cada endpoint del backend deba convertirse en una pantalla independiente.

1 Presentación del proyecto

Vas a diseñar e implementar la experiencia de usuario completa del mismo negocio que desarrollas en Entorno Cliente y Entorno Servidor. El objetivo de Diseño de Interfaces no es “decorar” el frontend al final: el diseño debe guiar desde el principio la estructura, los flujos, los estados de la aplicación y la forma en que se presentan los datos.

1.1 El ecosistema SafaReyes

SafaReyes es un complejo de ocio y servicios con cuatro negocios independientes que colaboran entre sí. Cada alumno trabaja uno de los cuatro casos, manteniendo la coherencia funcional con el backend común del proyecto.

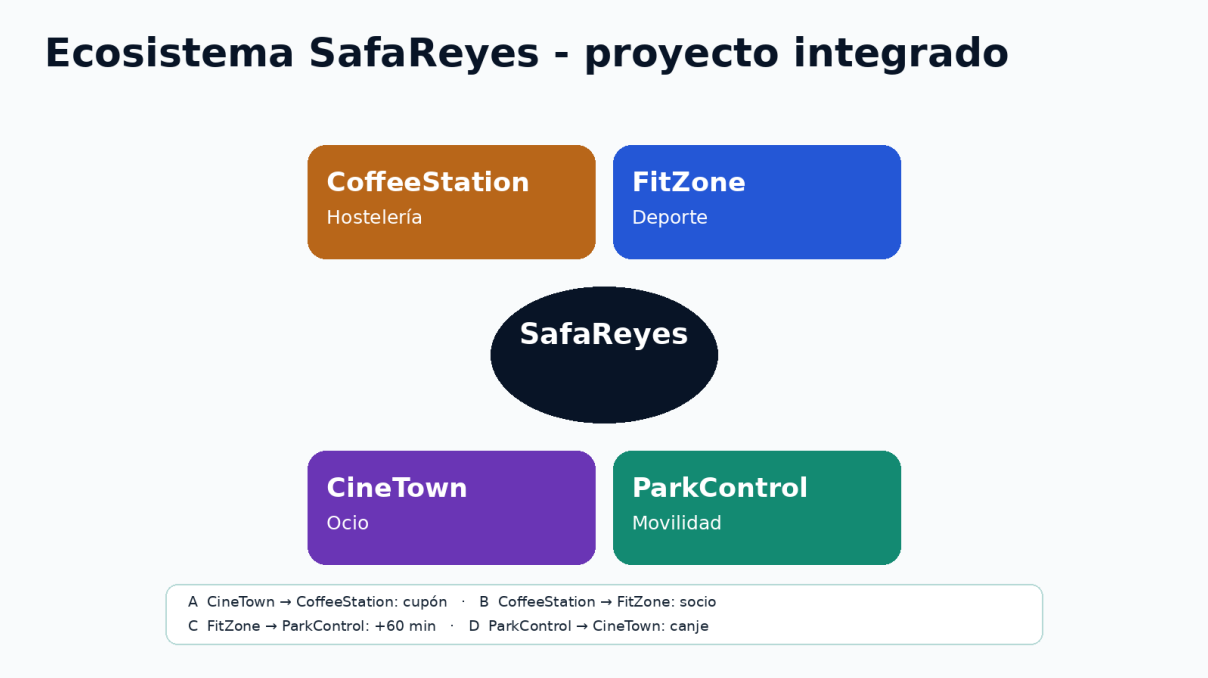


Figura 1. Cadena de integración del ecosistema SafaReyes.

1.2 Dinámica de trabajo

1. El caso asignado en Diseño de Interfaces será el mismo que en Entorno Cliente y Entorno Servidor.
2. El trabajo es individual. Puedes consultar documentación y comentar decisiones con compañeros, pero los artefactos de diseño y la implementación visual son propios.
3. El proyecto se construye de forma incremental. Primero se diseña con datos simulados; cuando la API esté disponible, el mismo diseño se conecta a datos reales.
4. No se evaluará en DIW la lógica de negocio interna del servidor, pero sí que la interfaz represente correctamente los estados, permisos, errores y reglas que el backend expone.
5. Las funcionalidades deben adaptarse al rol autenticado: la interfaz no mostrará acciones que ese perfil no puede realizar, sin sustituir nunca la seguridad real del servidor.

1.3 Qué se evalúa en esta asignatura

Bloque	Qué debe demostrar el proyecto
Planificación UI/UX	Problema, usuarios, arquitectura de información, flujos, wireframes, prototipo en Figma y justificación de decisiones.
Sistema visual	Guía de estilo, design tokens, tipografía, color, espaciado, componentes, estados y consistencia.
Implementación CSS	CSS/SCSS organizado, responsive, Grid/Flexbox, estilos alternativos, framework o librería acordada y validación.
Multimedia	Recursos con licencia, formatos apropiados, optimización, imágenes responsive y animaciones justificadas.
Accesibilidad	WCAG 2.2 AA como referencia: teclado, foco, contraste, semántica, formularios, mensajes y pruebas.
Usabilidad	Evaluación heurística, pruebas con usuarios, compatibilidad y mejora iterativa.

2 Resultados de aprendizaje y evidencias

El proyecto está planteado para generar evidencias directas de los resultados de aprendizaje de Diseño de Interfaces Web. Algunos criterios multimedia muy específicos (por ejemplo, edición avanzada de audio) pueden completarse con prácticas independientes cuando no tengan sentido real en el caso asignado.

RA	En el proyecto se traduce en…	Evidencias mínimas
RA1 · Planificación	Comunicación visual, color y tipografía, alternativas de presentación, guía de estilo, tecnologías y plantillas.	Brief UX, sitemap, user flows, wireframes, prototipo navegable en Figma y guía de estilo.
RA2 · Estilos	Estilos locales/globales, clases, propiedades, redefinición, hojas alternativas, validación, responsive, frameworks y preprocesadores.	SCSS/CSS modular, Grid/Flexbox, tema alternativo, framework/librería, validación y vistas responsive.
RA3 · Multimedia	Licencias, formatos, tratamiento y exportación de imagen/audio/vídeo y aplicación de guía de estilo.	Inventario de assets, optimización, formatos modernos, SVG y evidencia de tratamiento. Audio/vídeo cuando sea pertinente.
RA4 · Integración multimedia	Contenido multimedia/interactivo, código generado, interactividad y funcionamiento multidispositivo.	Componentes interactivos, microanimaciones, recursos responsive y pruebas en navegadores/dispositivos.
RA5 · Accesibilidad	Pautas, niveles de conformidad, errores, verificación, tecnologías y visibilidad/SEO.	Auditoría WCAG 2.2 AA, pruebas automáticas y manuales, SEO básico de vistas públicas.
RA6 · Usabilidad	Análisis, estándares, adecuación al usuario, navegación, técnicas de evaluación y compatibilidad.	Heurísticas, test con usuarios, métricas básicas, iteración y matriz de compatibilidad.

IMPORTANTE	No basta con que la aplicación “se vea bien”. Cada decisión relevante debe poder justificarse por un principio de comunicación visual, una necesidad del usuario, una restricción funcional o un criterio de accesibilidad/usabilidad.

3 Requisitos comunes de Diseño de Interfaces

3.1 Análisis UX y alcance

- Resumir el problema del cliente en una página: quién tiene el problema, qué necesita conseguir y qué valor aporta la aplicación.
- Definir al menos 2 perfiles/personas vinculados a los roles reales del caso. No inventar datos demográficos irrelevantes: priorizar objetivos, contexto, experiencia digital, necesidades y frustraciones.
- Definir al menos 3 tareas críticas del producto y su criterio de éxito. Ejemplo: “crear un pedido sin seleccionar un producto agotado”.
- Crear un inventario de contenidos y funcionalidades antes de diseñar pantallas.

3.2 Arquitectura de información y navegación

- Crear un sitemap con áreas públicas, autenticadas y específicas por rol.
- Diseñar al menos 3 user flows completos, incluyendo un camino feliz y al menos un error o alternativa.
- La navegación debe ser estable, comprensible y adaptada al rol. No se admiten menús con opciones inaccesibles para ese usuario.
- El usuario debe poder identificar dónde está, qué puede hacer y cómo volver sin depender del botón Atrás del navegador.

3.3 Wireframes, mockups y prototipo

FIGMA OBLIGATORIO. Todo el proceso de wireframing, mockup, componentes y prototipado navegable se desarrollará en Figma, salvo autorización expresa previa de la docente.

- Wireframes de baja o media fidelidad para todas las vistas principales antes de aplicar color y estilo final.
- Mockups de alta fidelidad de las vistas críticas en versión móvil y escritorio.
- Figma es obligatorio para el diseño y prototipado del proyecto. Se entregará un archivo/proyecto Figma accesible mediante enlace con permiso de visualización para la docente. El prototipo navegable debe permitir ejecutar las 3 tareas críticas e incluir las vistas y estados definidos. No se admiten Penpot, Canva u otras herramientas como sustitución salvo autorización expresa previa de la docente. Además del enlace se entregará una exportación PDF de respaldo; las capturas de pantalla por sí solas no sustituyen el archivo Figma.
- El prototipo debe incluir estados de componentes: normal, hover cuando proceda, foco, activo/seleccionado, deshabilitado, error y carga cuando sean relevantes.

3.4 Guía de estilo y sistema de componentes

Elemento	Requisito mínimo
Color	Paleta con roles: primary, secondary/accent, background, surface, text, success, warning y error. Contraste comprobado.
Tipografía	Familia(s), escala de títulos/cuerpo/auxiliares, pesos e interlineado. Máximo razonable de variantes.
Espaciado	Escala coherente (por ejemplo 4/8/12/16/24/32), no distancias arbitrarias por pantalla.
Componentes	Botones, inputs, selects, alertas, cards, navegación, tablas/listados, badges y modales si se usan.
Estados	Loading, vacío, éxito, error, deshabilitado, seleccionado y foco visible.
Tokens	Variables semánticas para color, espaciado, radios y, cuando proceda, sombras/tipografía.
Tema alternativo	Modo oscuro o alto contraste mediante variables/hoja alternativa. Debe conservar legibilidad y estados.

3.5 Implementación visual y CSS

- HTML semántico y estructura lógica. No utilizar tablas para maquetar ni elementos no semánticos cuando exista un control HTML apropiado.
- Estilos propios organizados en SCSS/Sass modular (salvo alternativa de preprocesador autorizada). Debe existir una salida CSS estándar consumida por el navegador.
- No se permite CSS inline como solución habitual. Los estilos puntuales solo se aceptan cuando estén técnicamente justificados.
- Utilizar Flexbox y CSS Grid con criterio. El layout debe ser fluido y no depender de posiciones absolutas para la estructura principal.
- Aplicar custom properties/tokens para decisiones globales. Cambiar el color de marca no debe exigir editar decenas de reglas.
- Utilizar y personalizar un framework CSS o librería de componentes aprobada en coordinación con Entorno Cliente. No se aceptará el aspecto por defecto sin adaptación al sistema visual.
- Validar el CSS y corregir errores relevantes. Los warnings aceptados deberán justificarse.

3.6 Responsive design

- Diseñar para un rango continuo de tamaños, no para modelos concretos de móvil. Como evidencia, se revisarán al menos 360 px, 768 px y 1440 px.
- Emplear unidades flexibles, Grid/Flexbox y media queries basadas en el contenido. Los breakpoints deben aparecer cuando el diseño lo necesita.
- No debe existir scroll horizontal accidental en las vistas ordinarias.
- Las tablas complejas deberán tener una estrategia móvil: reflujo, scroll controlado, tarjetas, prioridades de columnas u otra solución justificada.
- Las imágenes y contenedores multimedia no deben desbordar ni deformarse.

3.7 Estados asíncronos, permisos y errores

- Toda vista que dependa de datos remotos debe contemplar, cuando proceda: loading, vacío, éxito y error.
- Los errores 400/422 deben indicar qué campo o regla falla con lenguaje comprensible. No se mostrará al usuario el JSON técnico del servidor.
- 401 debe dirigir a autenticación o informar de sesión caducada; 403 debe explicar falta de permiso; 404, ausencia del recurso; 409, conflicto de estado/duplicado; 503, indisponibilidad temporal.
- Las acciones destructivas o irreversibles requieren confirmación o mecanismo equivalente cuando el riesgo lo justifique.
- Los mensajes no dependerán solo del color: combinar texto, iconografía y semántica cuando sea necesario.

3.8 Multimedia

- Mantener un inventario de recursos con origen/licencia. No utilizar material cuya licencia no permita el uso educativo/proyecto.
- Usar formatos adecuados: SVG para iconos/logos cuando corresponda; WebP/AVIF/JPEG/PNG según el contenido y compatibilidad requerida.
- Optimizar dimensiones y peso. No cargar una imagen de varios megapíxeles para mostrarla a 200 px.
- Implementar al menos una imagen responsive mediante srcset/sizes o <picture> cuando el contenido lo justifique.
- Incluir al menos una microinteracción o animación funcional y respetar prefers-reduced-motion.
- Audio y vídeo solo se incorporarán si aportan valor real al caso. Si no, esos criterios del RA se evidenciarán mediante práctica específica independiente.

3.9 Accesibilidad - referencia WCAG 2.2 AA

Área	Requisito
Estructura	Landmarks y encabezados lógicos; un h1 principal por vista cuando proceda; enlaces y botones con nombre accesible.
Teclado	Toda función debe poder operarse sin ratón. Orden de foco lógico, foco visible y sin trampas de teclado.
Contraste	Texto normal: 4.5:1; texto grande: 3:1 como referencia AA. Estados y controles deben seguir siendo distinguibles.
Formularios	Cada control con label, instrucciones cuando sean necesarias, errores vinculados al campo y resumen cuando el formulario sea complejo.
Imágenes	alt útil cuando informa; alt vacío cuando es decorativa. No repetir texto visible innecesariamente.
Tamaño objetivo	Objetivos de puntero de al menos 24×24 CSS px o que cumplan las excepciones de WCAG 2.2.
Movimiento	No depender de animación; respetar reduced motion; evitar parpadeos y animaciones molestas.
ARIA	Usar HTML semántico antes que ARIA. ARIA solo cuando mejora un patrón que HTML no resuelve por sí solo.

3.10 Usabilidad, compatibilidad y visibilidad

- Realizar una evaluación heurística de la interfaz utilizando como referencia las 10 heurísticas de Nielsen.
- Realizar una prueba de usabilidad con al menos 3 personas que no hayan diseñado la interfaz. Cada participante ejecutará al menos 2 tareas críticas.
- Registrar éxito/fracaso, tiempo aproximado, errores, dudas observadas y comentarios. No “ayudar” durante la tarea salvo bloqueo total.
- Aplicar al menos 3 mejoras verificables a partir de las pruebas, documentando antes/después.
- Probar la versión final en al menos dos motores de navegador de escritorio (Chromium y Firefox, por ejemplo) y en una viewport móvil realista.
- En vistas públicas: title descriptivo, meta description, estructura semántica de encabezados y enlaces comprensibles. Evitar texto importante incrustado en imágenes.

4 Entregables y estructura del repositorio

4.1 Entregables obligatorios

Fases	Entregable	Contenido
E1	Documento de análisis UX	Problema, alcance, usuarios, tareas críticas y decisiones iniciales.
E2	Arquitectura de información	Sitemap y 3 user flows, incluyendo errores/alternativas.
E3	Wireframes	Todas las vistas principales; móvil y escritorio para las críticas.
E4	Guía de estilo	Color, tipografía, espaciado, tokens, componentes y estados.
E5	Prototipo navegable en Figma	Enlace obligatorio al archivo Figma con permiso de visualización para la docente, prototipo conectado para las 3 tareas críticas y exportación PDF de respaldo. No se aceptan solo capturas.
E6	Implementación visual	Código integrado en el proyecto de Entorno Cliente, conectado al backend cuando corresponda.
E7	Informe de accesibilidad	Auditoría automática + pruebas manuales + correcciones.
E8	Informe de usabilidad	Heurísticas, test con 3 usuarios, resultados y mejoras.
E9	Evidencias finales	Capturas responsive, matriz de navegadores, recursos multimedia y checklist.

4.2 Estructura recomendada

docs/diseño/

```
docs/diseño/
├── 01-analisis-ux.pdf
├── 02-sitemap.pdf
├── 03-user-flows.pdf
├── 04-wireframes.pdf
├── 05-guia-estilo.pdf
├── 06-prototipo.md
├── 07-accesibilidad.pdf
├── 08-usabilidad.pdf
├── evidencias/
│   ├── responsive/
│   ├── navegadores/
│   └── auditorias/
└── assets/
    └── licencias.md
```

REPOSITORIO	La aplicación no debe duplicarse en un segundo frontend “solo para diseño”. La implementación visual evaluada en DIW es la misma que se entrega en Entorno Cliente. Los documentos de diseño sí se almacenan en docs/diseno/.

5 Fases y temporalización

La propuesta sigue el mismo horizonte de 10 semanas del proyecto backend, de forma que el diseño avance antes de necesitar la API real y pueda integrarse progresivamente.



Fase	Semanas	Contenido	Resultado esperado
F0 · Arranque y análisis	S1	Brief UX, roles, tareas críticas, inventario de contenidos.	Análisis UX aprobado.
F1 · Arquitectura	S1-2	Sitemap, navegación y 3 user flows.	Arquitectura y flujos validados.
F2 · Diseño	S2-3	Wireframes en Figma, guía de estilo, componentes, mockups y prototipo navegable.	Prototipo navegable en Figma aprobado antes de maquetar.
F3 · Implementación	S4-6	SCSS/CSS, framework/librería, responsive, estados y conexión progresiva.	Vistas principales implementadas y responsive.
F4 · Multimedia	S7	Optimización de recursos, imágenes responsive, microinteracciones y tema alternativo.	Assets optimizados y sistema visual completo.
F5 · Accesibilidad + UX	S8-9	Auditoría WCAG, teclado, compatibilidad, heurísticas y test con usuarios.	Informes y mejoras aplicadas.
F6 · Cierre	S10	Revisión final, evidencias, documentación y defensa.	Proyecto coherente, accesible, usable y documentado.

5.1 Lista de control por fase

F0 Caso y roles confirmados · 3 tareas críticas definidas · 2 perfiles/personas basados en roles reales

F1 Sitemap con áreas públicas/privadas · 3 user flows, uno con error · Navegación por rol definida

F2 Wireframes de vistas principales en Figma · Mockups móvil/escritorio de vistas críticas · Guía de estilo, componentes y prototipo navegable en Figma

F3 SCSS/CSS modular · Grid/Flexbox y responsive · Estados loading/vacío/error/éxito · Framework/librería personalizado

F4 Assets optimizados y licenciados · Tema alternativo · Microinteracción y reduced motion

F5 Auditoría automática · Prueba de teclado y zoom · Test con 3 usuarios · 3 mejoras aplicadas

F6 Documentos finales · Capturas y matriz de compatibilidad · Checklist completa · Defensa preparada

CASO 1

CoffeeStation

Hostelería · catálogo, pedidos y cupones

6.1 Roles y navegación

Rol/área	Necesidad de interfaz
Público	Consultar carta y detalles.
CLIENTE	Crear pedidos propios, verlos y validar descuentos.
BARISTA	Gestionar cola de pedidos, estados, disponibilidad y cupones.
ADMIN	Gestionar carta, cupones e informes.

6.2 Vistas mínimas

Fase	Área	Vista	Requisito UI/UX
1	Pública	Carta / catálogo	Búsqueda y filtros por categoría, texto, precio, disponibilidad y alérgenos. Estado agotado claramente visible.
2	Pública	Detalle de producto	Descripción, precio, categoría, disponibilidad y alérgenos; CTA coherente con estado.
3	Cliente	Constructor de pedido	Líneas, cantidades, validación de agotados, subtotal y feedback inmediato.
4	Cliente	Descuentos y confirmación	Cupón o DNI de socio; mostrar el descuento aplicado y explicar por qué se usa uno u otro.
5	Cliente	Seguimiento / historial	Turno, estado del pedido, líneas e importes; solo pedidos propios.
6	Barista	Cola de pedidos	Prioridad por estado/fecha, cambios de estado y feedback de operación.
7	Barista	Disponibilidad	Agotar/reponer producto y categoría sin confundir baja lógica con agotado.
8	Admin	Gestión de carta	Alta/edición/baja lógica de productos, categorías y alérgenos.
9	Admin	Cupones	Creación, vigencia, límites de uso, personalización y validación.
10	Admin	Informes	Ventas del día y ranking de productos con visualización comprensible.

6.3 Flujos críticos

1) Cliente crea un pedido con producto disponible y recibe número de turno. **2)** Cliente intenta usar cupón no válido o descuento de socio; la interfaz explica el resultado. **3)** Barista avanza PENDIENTE → EN_PREPARACION → LISTO → ENTREGADO.

6.4 Casos límite y aceptación

Diseñar explícitamente: Producto agotado o retirado · Cupón caducado/agotado/personal no aplicable · FitZone no disponible: el pedido continúa sin descuento de socio y se informa · Pedido no cancelable por estado

Criterio de aceptación: las funciones principales deben tener representación usable, responsive y coherente con roles y reglas; no es obligatorio crear una pantalla aislada por cada endpoint.

10 Contratos de interfaz con Cliente y Servidor

El proyecto integrado funciona como un único producto. La capa de interfaz debe anticipar y representar correctamente lo que ocurre tanto en Entorno Cliente como en Entorno Servidor.

10.1 Contrato con Desarrollo Web en Entorno Servidor

- La interfaz consumirá los endpoints y modelos definidos por el backend del caso. No se cambiarán nombres o reglas del contrato desde DIW para “hacer más fácil” una pantalla.
- Los DTOs y ProblemDetail del servidor se transformarán en mensajes de usuario adecuados. No se debe mostrar información técnica, stack traces ni estructuras JSON.
- Las restricciones de negocio deben anticiparse visualmente cuando sea posible: deshabilitar acciones imposibles, explicar condiciones y prevenir errores antes de enviar.
- La UI debe reflejar los roles del backend, pero nunca sustituir la autorización del servidor.

10.2 Estados HTTP y tratamiento visual

Respuesta	Tratamiento esperado en UI
200/201	Confirmación visible y actualización de la vista sin obligar a recargar manualmente.
204	Eliminar/cerrar el recurso visual correspondiente y confirmar la acción cuando sea relevante.
400	Errores de formato/validación cerca del campo y resumen cuando haya varios.
401	Sesión ausente/caducada: informar y conducir a autenticación conservando contexto cuando sea posible.
403	Explicar que el usuario no tiene permiso; no presentarlo como “recurso inexistente” salvo decisión de seguridad del producto.
404	Estado vacío/ausente con una salida clara: volver, buscar o crear según el caso.
409	Conflicto: explicar duplicado o estado incompatible y qué puede hacer el usuario.
422	Regla de negocio: explicar la condición que impide continuar.
503	Servicio no disponible: informar de forma no destructiva y seguir la regla del backend sobre si la operación principal continúa.

10.3 Integraciones entre negocios

Contrato	Desde la perspectiva del usuario
A · CineTown → CoffeeStation	Tras comprar entradas, mostrar si el cupón de cafetería se generó. Si falla, la compra de cine sigue siendo válida.
B · CoffeeStation → FitZone	Al aplicar DNI de socio, indicar si se obtuvo descuento. Si FitZone no responde, el pedido sigue sin ese descuento y debe explicarse.
C · FitZone → ParkControl	Tras entrada permitida, la bonificación de parking es secundaria. Si falla, no se bloquea el acceso al gimnasio.
D · ParkControl → CineTown	Al pagar parking con código de cine, indicar si se validó/canjeó. Si no, el cobro continúa sin esa promoción.

ESTADO DE INTEGRACIÓN	Si el backend devuelve estados como OK, NO_APLICA, RECHAZADO o NO_DISPONIBLE, la interfaz debe traducirlos a mensajes comprensibles. El nombre técnico del estado puede permanecer en logs o herramientas de desarrollo, no tiene por qué ser el texto mostrado al usuario.

10.4 Desarrollo sin depender del backend

1. Diseñar y prototipar antes de que la API esté terminada.
2. Usar datos mock con la misma forma contractual que el backend para desarrollar estados y componentes.
3. Probar también errores y demoras, no solo respuestas correctas.
4. Sustituir el mock por la API real sin rediseñar la interfaz: si el cambio obliga a rehacer pantallas, probablemente el contrato visual no estaba bien definido.

DOCUMENTO 3 DE 3

DWES · Entorno Servidor

Se incluyen la arquitectura y requisitos técnicos comunes, el caso completo de CoffeeStation y los contratos de integración que afectan a su API.

A continuación se incluyen las páginas seleccionadas del documento original.

2º DAW · Desarrollo Web en Entorno Servidor

Curso 2026/2027

PROYECTO INTEGRADO INDIVIDUAL

Backend con Spring Boot

Ecosistema **SafaReyes**

De las necesidades de un cliente real a una API REST segura: base de datos en PostgreSQL, persistencia con Spring Data JPA, DTOs con MapStruct, seguridad con JWT e integración entre servicios.

CASO 1
CoffeeStation
Hostelería

CASO 2
FitZone
Deporte y control de accesos

CASO 3
CineTown
Ocio y venta de entradas

CASO 4
ParkControl
Movilidad y estancias

Java 21 · Spring Boot 3 · PostgreSQL 16 · JPA · Lombok · MapStruct · Spring Security + JWT · RestClient

Contenido

1 Presentación del proyecto 1

2 Stack tecnológico y arquitectura 3

3 Requisitos técnicos comunes 6

4 Fases y temporalización 9

5 Caso 1 · CoffeeStation 11

6 Caso 2 · FitZone 15

7 Caso 3 · CineTown 19

8 Caso 4 · ParkControl 23

9 Contratos de integración 27

Cómo usar este documento

Lee completas las secciones **1 a 4** y la **9**: son comunes para todo el grupo. De las secciones 5 a 8 solo necesitas la de tu caso y la del caso con el que te integras.

Cada caso sigue la misma estructura: reunión con el cliente, roles, requisitos funcionales (RF), reglas de negocio (RN), entidades mínimas, los 16 endpoints, sugerencias de consultas e integración.

1 Presentación del proyecto

Vas a desarrollar, de forma individual, el backend completo de un negocio real: desde el diseño de su base de datos en PostgreSQL hasta una API REST segura con Spring Boot que, en la fase final, se comunica con la API de un compañero.

1.1 El ecosistema SafaReyes

SafaReyes es un complejo de ocio y servicios de nueva apertura. En su interior conviven cuatro negocios independientes que han decidido digitalizarse al mismo tiempo y colaborar entre sí para fidelizar a sus clientes.

CASO 1	CASO 2	CASO 3	CASO 4
CoffeeStation	**FitZone**	**CineTown**	**ParkControl**
Hostelería	Deporte y control de accesos	Ocio y venta de entradas	Movilidad y estancias
Catálogo, pedidos en barra y cupones	Socios, cuotas, torno de entrada y actividades	Cartelera, salas, sesiones y compras	Plazas, tickets de estancia, tarifas y abonados

Cada negocio tiene su propio sistema, desarrollado por ti de forma independiente. Pero los cuatro forman una **cadena de colaboración**: cada API **consume** un servicio de otra y **expone** un servicio para otra. La flecha va del sistema que llama (cliente HTTP) al que responde (servidor). Así, todos los casos hacen el mismo trabajo de integración: un endpoint expuesto y un cliente HTTP.

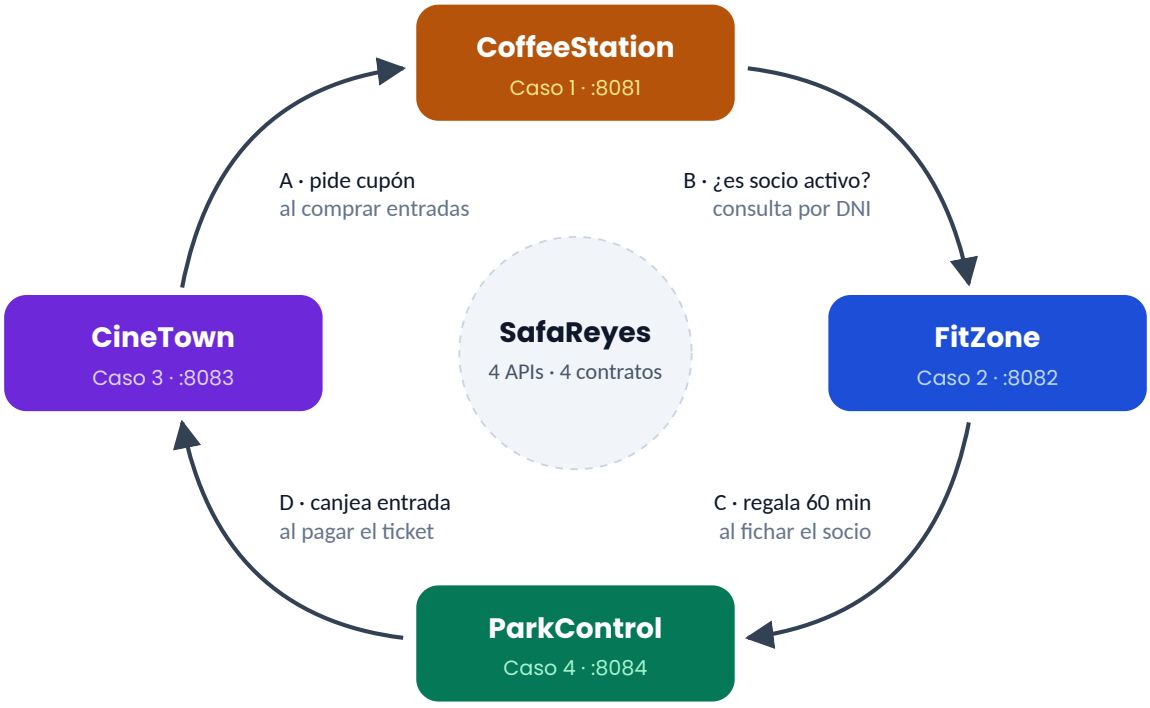


Figura 1. Cadena de integración del ecosistema SafaReyes

1.2 Dinámica de trabajo

1. El profesor te asigna uno de los cuatro casos. Si hay más alumnos que casos, varios alumnos desarrollarán el mismo caso de forma independiente.

2. Actúas como desarrollador principal: eres responsable del diseño de datos, la lógica de negocio, la seguridad y la API.
3. El enunciado de tu caso está escrito como lo contaría el cliente. Tu primera tarea es **analizarlo**: extraer entidades, atributos, relaciones y reglas. La base de datos no se te da hecha.
4. En la Fase 5 el profesor te asigna un **compañero de integración** del caso al que tu API llama. Los contratos (URL, JSON, cabeceras) están fijados en la sección 9 y son idénticos para todos, así que puedes desarrollar tu cliente antes de que tu compañero termine.
5. El trabajo es individual. Puedes consultar documentación y resolver dudas con compañeros, pero el código y el diseño son tuyos.

1.3 Qué debes leer

Las secciones 2, 3, 4 y 9 son comunes y obligatorias para todos. De las secciones 5 a 8 solo necesitas la de tu caso, aunque te conviene leer también la del caso con el que te integras.

2 Stack tecnológico y arquitectura

Todos los proyectos usan exactamente el mismo stack. No se admiten sustituciones sin autorización previa del profesor.

2.1 Tecnologías obligatorias

Área	Tecnología	Uso en el proyecto
Lenguaje	Java 21	Records para DTOs, `switch` moderno, `Optional`
Framework	Spring Boot 3.x (última estable)	Proyecto generado con Spring Initializr y Maven
Base de datos	PostgreSQL 16	Arrancada con Docker Compose (`compose.yaml` en el repositorio)
Persistencia	Spring Data JPA + Hibernate	Entidades, relaciones y repositorios
Validación	Jakarta Bean Validation	`@Valid` , `@NotBlank` , `@Positive` , `@Email` , validadores propios
Boilerplate	Lombok	Entidades, servicios, logging
Mapeo	MapStruct	Conversión entidad ↔ DTO, sin mapeo manual
Seguridad	Spring Security 6 + JWT	Autenticación sin estado y autorización por roles
Cliente HTTP	`RestClient` de Spring	Consumo de la API del compañero
Documentación	springdoc-openapi (Swagger UI)	Documentación viva de todos los endpoints
Pruebas manuales	Postman o Bruno	Colección exportada en el repositorio
Control de versiones	Git + GitHub	Repositorio del proyecto

Dependencias de Initializr: *Spring Web*, *Spring Data JPA*, *PostgreSQL Driver*, *Validation*, *Spring Security*, *Lombok*, *Spring Boot DevTools*. MapStruct, JJWT y springdoc se añaden a mano en el `pom.xml` .

> **Importante** Lombok debe ejecutarse antes que MapStruct. Configura `maven-compiler-plugin` con `lombok` , `lombok-mapstruct-binding` y `mapstruct-processor` en `annotationProcessorPaths` .

2.2 Arquitectura en capas

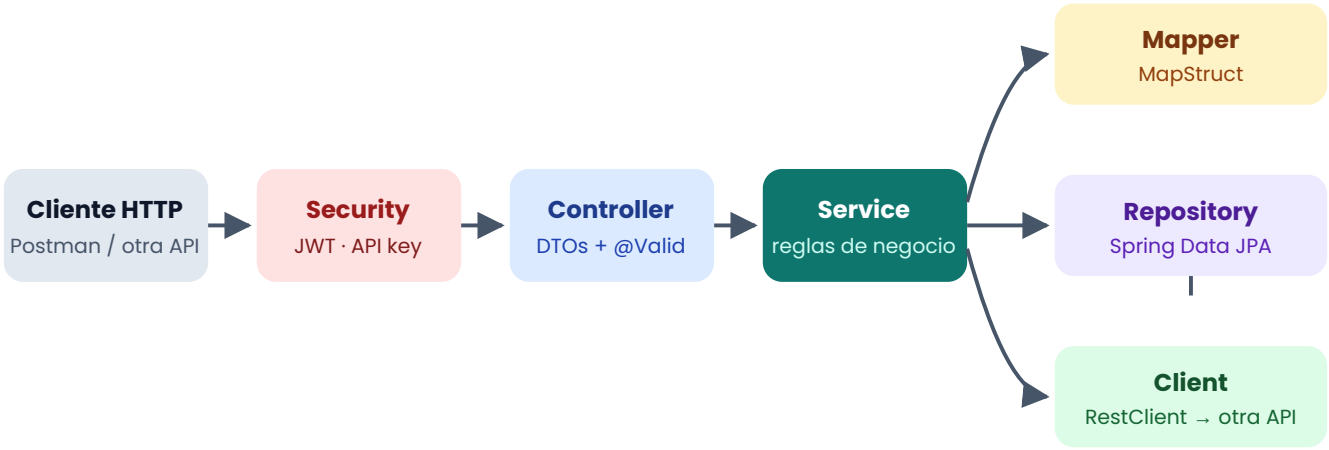


Figura 2. Capas de la aplicación y sentido de las dependencias

- Un controlador nunca accede a un repositorio ni devuelve una entidad. Recibe y devuelve DTOs.
- Toda la lógica de negocio vive en la capa de servicio. Las operaciones que escriben usan `@Transactional`.
- La inyección de dependencias es siempre por constructor (`@RequiredArgsConstructor` + campos `final`). Prohibido `@Autowired` en atributos.
- Las llamadas a la API externa se encapsulan en una clase de la capa `client`. El servicio no conoce URLs ni cabeceras.

2.3 Estructura de paquetes

PAQUETES

```
es.safareyes.<caso>
├── config          → RestClient, OpenAPI, propiedades
├── controller      → @RestController
├── dto
│   ├── request     → records de entrada
│   └── response    → records de salida
├── entity          → @Entity y enums
├── exception       → excepciones propias + @RestControllerAdvice
├── mapper          → interfaces @Mapper (MapStruct)
├── repository      → interfaces JpaRepository
├── security        → SecurityConfig, filtro JWT, UserDetailsService
├── service         → interfaces + implementaciones
└── client          → cliente de integración con la otra API
```

Sustituye `<caso>` por `coffeestation`, `fitzone`, `cinetown` o `parkcontrol`. Nombre recomendado del repositorio: `safareyes-<caso>-<tu-apellido>`.

2.4 Convenciones

- Tablas y columnas en `snake_case` y plural para tablas (`lineas_pedido`). Clases en `PascalCase`, atributos en `camelCase`.
- Todas las rutas empiezan por `/api/v1`, usan sustantivos en plural y minúsculas con guiones (`/informes/ventas-dia`).
- Importes con `BigDecimal` en Java y `NUMERIC(10,2)` en PostgreSQL. Nunca `double` para dinero.
- Fechas con `LocalDate`, `LocalDateTime` y zona `Europe/Madrid`.

- Configuración sensible (contraseña de BD, secreto JWT, API keys) en variables de entorno, nunca en el código subido a GitHub.
- Commits pequeños y con mensajes descriptivos.

3 Requisitos técnicos comunes

Estos requisitos son iguales en los cuatro casos. En la ficha de cada caso encontrarás ejemplos concretos de dónde aplicarlos.

3.1 Base de datos

- Mínimo **6 tablas de dominio** más las tablas de seguridad (`usuarios` y, si lo necesitas, `roles`).
- Modelo normalizado hasta 3FN. Justifica en el README cualquier desnormalización (por ejemplo, guardar el precio aplicado en una línea).
- Al menos una relación **1:N**, una **N:M** con tabla intermedia y una relación opcional (FK que admite `NULL`).
- Restricciones declaradas en el script: `PRIMARY KEY`, `FOREIGN KEY` con su `ON DELETE`, `UNIQUE`, `NOT NULL`, `CHECK` (importes positivos, rangos, estados válidos).
- Al menos un índice creado a mano sobre una columna por la que se busca con frecuencia.
- `data.sql` con datos suficientes para probar todos los endpoints: mínimo 10 filas en las tablas principales y casos límite (agotados, caducados, inactivos, completos).

3.2 Entidades y Lombok

- Anotaciones Lombok permitidas en entidades: `@Getter`, `@Setter`, `@NoArgsConstructor`, `@AllArgsConstructor`, `@Builder`, `@ToString(exclude = …)`.
- **Prohibido `@Data` en entidades**: su `equals`, `hashCode` y `toString` recorren las relaciones y provocan bucles y cargas innecesarias. Debes saber explicar por qué.
- En servicios y controladores: `@RequiredArgsConstructor` y `@Slf4j`. Registra en log las operaciones de negocio relevantes y los fallos de integración.
- Los enums se persisten con `@Enumerated(EnumType.STRING)`.
- Relaciones con `FetchType.LAZY` por defecto. Usa `EAGER` solo si lo justificas.
- Al menos una entidad con campos de auditoría (`creadoEn`, `actualizadoEn`) rellenos automáticamente con `@PrePersist` / `@PreUpdate` o con JPA Auditing.

3.3 Formatos de consulta en repositorios

Este es el núcleo de la Fase 2. Tus repositorios deben contener, como mínimo, las consultas de esta tabla, y todas deben usarse desde algún servicio.

#	Formato	Mín.	Qué debe demostrar
Q1	Métodos derivados del nombre (*query methods*)	5	Combinar `And` / `Or`, `Between`, `LessThan` / `GreaterThan`, `Containing` / `IgnoreCase`, `OrderBy`, `existsBy`, `countBy`, `findFirst` / `Top`
Q2	JPQL con `@Query` y parámetros con nombre (`@Param`)	3	Al menos una con `JOIN` entre entidades y una con función de agregado (`SUM`, `COUNT`, `AVG`) y `GROUP BY`
Q3	SQL nativo con `@Query(nativeQuery = true)`	2	Consultas de informe con funciones propias de PostgreSQL (`DATE_TRUNC`, `EXTRACT`, `COALESCE`…)
Q4	Consultas de modificación con `@Modifying`	1	`UPDATE` o `DELETE` masivo, dentro de una transacción, con `clearAutomatically = true`

#	Formato	Mín.	Qué debe demostrar
Q5	Proyecciones	2	Una proyección por **interfaz** y una por **clase/record** con `SELECT new …` en JPQL
Q6	Paginación y ordenación	1	Parámetro `Pageable` y retorno `Page<…>`, expuesto en un endpoint con `?page=&size=&sort=`
Q7	Consultas dinámicas	1	`JpaSpecificationExecutor` + `Specification` para filtros opcionales combinables
Q8	Evitar el problema N+1	1	`@EntityGraph` o `JOIN FETCH` en una consulta que cargue una entidad con su colección

16 consultas propias como mínimo. En el README incluye una tabla con cada consulta, su formato (Q1–Q8), el repositorio en que está y el endpoint que la usa.

3.4 DTOs y MapStruct

- Todos los DTOs son `record` de Java, separados en `dto.request` y `dto.response`.
- Cada recurso tiene como mínimo un DTO de creación, uno de respuesta y, si se puede modificar, uno de actualización. Los DTOs de respuesta nunca incluyen contraseñas ni datos internos.
- Validaciones de entrada en los DTOs de petición, nunca en las entidades. Al menos un **validador personalizado** (anotación propia + `ConstraintValidator`), por ejemplo para DNI, matrícula o código de cupón.
- Mapeo exclusivamente con MapStruct (`@Mapper(componentModel = "spring")`).

Los mappers deben demostrar:

- `@Mapping(target = …, source = …)` con campos de nombre distinto o anidados (`source = "sala.numero"`).
- `@Mapping(target = …, ignore = true)` para campos que calcula el servicio.
- Actualización parcial con `@MappingTarget` y `NullValuePropertyMappingStrategy.IGNORE`.
- Mapeo de colecciones (lista de entidades a lista de DTOs) y uso de otro mapper con `uses = …`.

3.5 Seguridad

- Autenticación **sin estado** con JWT. Endpoints comunes a todos los casos: `POST /api/v1/auth/registro` (alta de un usuario con rol de cliente, socio o abonado) y `POST /api/v1/auth/login` (devuelve el token JWT, que caduca en 60 minutos).
- Contraseñas cifradas con `BCryptPasswordEncoder`.
- Tres roles por caso (definidos en tu ficha). Los usuarios con roles de personal se crean desde `data.sql` o por un administrador.
- Autorización en dos niveles: reglas por ruta en el `SecurityFilterChain` y `@PreAuthorize` en al menos tres métodos.
- Al menos una regla de **propiedad del recurso**: un cliente solo puede consultar sus propios pedidos, compras, cuotas o tickets.
- Los endpoints de integración (sección 9) no usan JWT: se protegen con la cabecera `X-API-KEY`, comprobada por un filtro propio. La clave se lee de una variable de entorno.
- Swagger UI y `/api/v1/auth/**` son públicos. CSRF desactivado por ser una API sin estado.

3.6 Gestión de errores y códigos HTTP

Todos los errores se devuelven con el mismo formato, basado en `ProblemDetail` (RFC 9457), generado desde un único `@RestControllerAdvice`. Crea al menos cuatro excepciones propias (por ejemplo `RecursoNoEncontradoException`, `ReglaNegocioException`, `ConflictoException`, `IntegracionException`).

JSON

```
{
  "type": "about:blank",
  "title": "Recurso no encontrado",
  "status": 404,
  "detail": "No existe el producto con id 57",
  "instance": "/api/v1/productos/57",
  "timestamp": "2026-10-14T09:31:02"
}
```

Código	Cuándo usarlo
200 OK	Consulta o modificación correcta
201 Created	Recurso creado, con cabecera `Location`
204 No Content	Borrado (lógico o físico) correcto
400 Bad Request	Validación de DTO fallida (incluye la lista de campos erróneos)
401 Unauthorized	Sin token, token inválido o caducado, API key incorrecta
403 Forbidden	Token válido pero rol insuficiente o recurso ajeno
404 Not Found	El recurso no existe
409 Conflict	Duplicados o estado incompatible (DNI repetido, cupón ya canjeado)
422 Unprocessable Content	La petición es correcta pero viola una regla de negocio (sesión completa, sin plazas)
503 Service Unavailable	Solo en el cliente de integración, si la otra API no responde y la operación no puede continuar

3.7 Documentación y calidad

- Swagger UI accesible en `/swagger-ui.html` con cada endpoint descrito (`@Operation`, `@Tag`) y el esquema de seguridad Bearer configurado.
- README con: descripción, instrucciones para arrancar con Docker Compose, variables de entorno, diagrama E/R, usuarios de prueba, tabla de consultas (3.3) y decisiones de diseño.
- Colección de Postman o Bruno con todas las peticiones, incluidos casos de error.
- Pruebas automáticas (recomendado): tests de repositorio con `@DataJpaTest` y de servicio con JUnit 5 + Mockito.
- El proyecto debe arrancar con `docker compose up -d` y `./mvnw spring-boot:run` siguiendo solo el README.

4 Fases y temporalización

El proyecto se organiza en **10 semanas** repartidas en 7 fases (de la 0 a la 6). Las fechas concretas y el procedimiento de entrega de cada fase los indicará el profesor.



Fase	Semanas	Contenido	Resultado esperado
0 · Arranque y análisis	1	Lectura del caso, repositorio, Docker Compose	Proyecto creado y conectado a PostgreSQL
1 · Base de datos	1–2	Modelo E/R, modelo relacional, scripts	`schema.sql`, `data.sql`, diagrama E/R y documento de análisis
2 · Persistencia JPA	3–4	Entidades, relaciones, repositorios y consultas	Entidades mapeadas y las 16 consultas funcionando
3 · API REST	5–6	Servicios, DTOs, MapStruct, controladores, errores	Los 16 endpoints de negocio de tu caso
4 · Seguridad	7	Usuarios, JWT, roles, API key	API protegida según la matriz de roles de tu caso
5 · Integración	8–9	Endpoint expuesto y cliente `RestClient`	Integración probada con tu compañero
6 · Cierre	10	Revisión final y documentación	Proyecto completo y documentado

Lista de tareas por fase

Fase 0 · Arranque y análisis

- [ ] Crear el repositorio Git `safareyes-<caso>-<tu-apellido>`.
- [ ] Generar el proyecto en Spring Initializr con las dependencias de la sección 2.1.
- [ ] Añadir `compose.yaml` con PostgreSQL 16 y comprobar que la aplicación conecta.
- [ ] Leer tu caso y subrayar sustantivos (posibles entidades), verbos (operaciones) y condiciones (reglas de negocio).

Fase 1 · Base de datos

- [ ] Documento `docs/analisis.md`: entidades, atributos, relaciones con su cardinalidad y reglas de negocio que afectan a los datos.
- [ ] Diagrama E/R (draw.io, dbdiagram.io o similar) exportado en `docs/`.
- [ ] `schema.sql` con todas las restricciones de la sección 3.1.
- [ ] `data.sql` con datos realistas y casos límite.
- [ ] Tres consultas SQL de ejemplo en `docs/consultas.sql` que respondan a preguntas del cliente (base para las consultas nativas).

En esta fase la base de datos se crea con los scripts. A partir de la Fase 2 configura `spring.jpa.hibernate.ddl-auto=validate` para que Hibernate compruebe que tus entidades coinciden con tu esquema.

Fase 2 · Persistencia JPA

- [ ] Una entidad por tabla de dominio, con relaciones bidireccionales donde tenga sentido (`mappedBy`).
- [ ] Enums del dominio y campos de auditoría (sección 3.2).
- [ ] Repositorios con las 16 consultas mínimas de la sección 3.3.
- [ ] `CommandLineRunner` temporal o tests `@DataJpaTest` que demuestren que cada consulta funciona.

Fase 3 · API REST

- [ ] DTOs y mappers MapStruct según la sección 3.4.
- [ ] Servicios con todas las reglas de negocio (RN) de tu caso.
- [ ] Controladores con los 16 endpoints de negocio de tu caso, códigos HTTP correctos y paginación.
- [ ] `@RestControllerAdvice` y excepciones propias (sección 3.6).
- [ ] Swagger UI funcionando.

Fase 4 · Seguridad

- [ ] Tabla `usuarios`, entidad y `UserDetailsService`.
- [ ] Registro, login y filtro JWT.
- [ ] Matriz de roles de tu caso aplicada y regla de propiedad del recurso.
- [ ] Filtro de API key para `/api/v1/integracion/**`.

Fase 5 · Integración

- [ ] Implementar el endpoint que **expones** según el contrato de la sección 9.
- [ ] Implementar el cliente `RestClient` que **consume** la API de tu compañero, con URL y API key en `application.yml` leídas de variables de entorno.
- [ ] Tiempo de espera máximo de 3 segundos y tratamiento de errores (4xx, 5xx, sin respuesta).
- [ ] Desarrollar primero contra un simulador y después contra la API real del compañero.
- [ ] Prueba conjunta con tu compañero documentada en `docs/integracion.md`.

Fase 6 · Cierre

- [ ] README completo, colección Postman/Bruno y Swagger actualizados.
- [ ] Repasar los requisitos de las secciones 3 y de tu caso y comprobar que se cumplen todos.
- [ ] Comprobar que el proyecto arranca desde cero siguiendo solo el README.

CASO 1

CoffeeStation

Hostelería

CoffeeStation es la cafetería de la planta baja de SafaReyes. Necesita una API que gestione su carta, los pedidos en barra y un sistema de cupones de descuento.

5.1 Reunión con el cliente

Laura Méndez, gerente de CoffeeStation

«Abrimos a las siete y media y entre las ocho y las diez la barra es un caos. Apuntamos las comandas en papel, se pierden, y cuando el cliente pregunta cuánto le debe, el camarero suma de cabeza. Queremos un sistema que lo ordene todo.

Lo primero es la carta. Tenemos productos organizados por categorías: cafés, tés e infusiones, bollería, bocadillos, zumos y una línea saludable. De cada producto necesito el nombre, una descripción corta, el precio de venta con IVA incluido, si ahora mismo está disponible o agotado y qué alérgenos contiene, porque nos lo exige sanidad y nos lo preguntan a diario. Un producto puede tener varios alérgenos y un mismo alérgeno está en muchos productos. Cuando retiramos algo de la carta no quiero perderlo, porque aparece en ventas antiguas: simplemente deja de mostrarse.

A veces se nos acaba toda la bollería a media mañana. Me gustaría que el barista pudiera marcar de golpe toda una categoría como agotada en lugar de ir producto a producto.

Los pedidos los toma el barista en la tablet, o los hace el propio cliente desde la app si está registrado. Un pedido tiene varios productos con su cantidad. Al crearlo se le da un número de turno que empieza en 1 cada día, para llamar al cliente cuando esté listo. El pedido pasa por pendiente, en preparación, listo y entregado. Si el cliente se arrepiente, solo se puede cancelar mientras esté pendiente. Y muy importante: si mañana subo el precio del café, los pedidos de ayer tienen que seguir mostrando lo que se cobró.

Queremos lanzar cupones. Algunos son de porcentaje, como un 10 %, y otros de importe fijo, como 1 euro. Cada cupón tiene un código, fechas de validez, un número máximo de usos y a veces un importe mínimo de pedido. Algunos cupones son personales: solo los puede usar el cliente cuyo email tiene asignado. Antes de cobrar, el barista tiene que poder comprobar si un código es válido y cuánto descuenta.

Por último, los socios del gimnasio FitZone del complejo tienen un 10 % de descuento con nosotros. Ahora mismo nos enseñan el carné y nos fiamos. Queremos que el sistema pregunte al gimnasio si ese DNI es de un socio activo. El descuento de socio no se acumula con un cupón: se aplica el que más le convenga al cliente.

Al cierre quiero ver cuánto hemos vendido en el día, desglosando base e IVA, y un ranking de los productos más vendidos entre dos fechas.»

5.2 Actores y roles

Rol	Quién es	Qué puede hacer
`ADMIN`	Gerencia	Todo: carta, cupones, informes, gestión de pedidos
`BARISTA`	Personal de barra	Crear y gestionar pedidos, cambiar disponibilidad, validar cupones
`CLIENTE`	Cliente registrado en la app	Consultar la carta, crear pedidos propios, ver solo sus pedidos, validar cupones

5.3 Requisitos funcionales

RF-01
Catálogo
Alta, modificación, consulta y baja lógica de productos con categoría, precio, disponibilidad y alérgenos.

RF-02
Filtros de carta
Buscar productos por categoría, texto en el nombre, precio máximo, disponibilidad y excluyendo un alérgeno, con paginación.

RF-03
Agotado por categoría
Marcar como disponibles o agotados todos los productos de una categoría en una sola operación.

RF-04
Pedidos
Crear pedidos con líneas, calcular importes, asignar número de turno diario y aplicar un cupón o el descuento de socio.

RF-05
Ciclo del pedido
Cambiar el estado de un pedido respetando las transiciones permitidas.

RF-06
Cupones
Crear cupones y validar un código para un importe dado, devolviendo el descuento resultante.

RF-07
Informes
Ventas del día y productos más vendidos entre dos fechas.

RF-08
Integración
Consultar a FitZone si un DNI es de un socio activo y generar cupones a petición de CineTown.

5.4 Reglas de negocio

RN-01 Un producto dado de baja no aparece en la carta ni puede pedirse, pero se conserva en los pedidos antiguos.

RN-02 Pedir un producto agotado o dado de baja devuelve 422 indicando qué producto falla.

RN-03 Cada línea guarda el precio unitario vigente al crear el pedido. Los cambios de precio posteriores no afectan a pedidos existentes.

RN-04 Un pedido tiene entre 1 y 15 líneas y cada línea entre 1 y 20 unidades. Si el mismo producto llega en dos líneas, se agrupan en una.

RN-05 Los precios incluyen IVA del 10 %. El pedido guarda subtotal, descuento, total, base imponible (total / 1,10) y cuota de IVA, redondeados a 2 decimales.

RN-06 Un cupón es válido si existe, está activo, la fecha actual está dentro de su vigencia, no ha alcanzado sus usos máximos, el subtotal supera su importe mínimo y, si es personal, el email coincide con el del cliente.

RN-07 El descuento de socio FitZone (10 %) y un cupón no se acumulan: se aplica el mayor. El total nunca puede ser negativo.

RN-08 Transiciones válidas: `PENDIENTE → EN_PREPARACION → LISTO → ENTREGADO` y `PENDIENTE → CANCELADO`. Cualquier otra devuelve 422. Cancelar un pedido devuelve el uso al cupón aplicado.

RN-09 El número de turno es el número de pedidos creados ese día más uno.

RN-10 Los informes de ventas solo cuentan pedidos `ENTREGADO`.

5.5 Entidades mínimas

`Categoria` `Producto` `Alergeno` `Pedido` `LineaPedido` `Cupon` `Usuario`

La relación producto–alérgeno es N:M. Un pedido puede tener un cupón y un cliente (ambos opcionales). Deduce los atributos a partir de la reunión y de las reglas.

5.6 Endpoints de negocio (16)

#	Método	Ruta	Descripción	Roles
1	GET	`/api/v1/productos`	Carta paginada con filtros opcionales (RF-02)	Público
2	GET	`/api/v1/productos/{id}`	Detalle con alérgenos	Público
3	POST	`/api/v1/productos`	Alta de producto	ADMIN
4	PUT	`/api/v1/productos/{id}`	Modificación de producto	ADMIN
5	PATCH	`/api/v1/productos/{id}/disponibilidad`	Marcar disponible o agotado	ADMIN, BARISTA
6	DELETE	`/api/v1/productos/{id}`	Baja lógica	ADMIN
7	GET	`/api/v1/categorias`	Categorías con número de productos activos	Público
8	PATCH	`/api/v1/categorias/{id}/disponibilidad`	Agotar o reponer toda una categoría	ADMIN, BARISTA
9	POST	`/api/v1/pedidos`	Crear pedido (con `codigoCupon` o `dniSocio` opcionales)	BARISTA, CLIENTE
10	GET	`/api/v1/pedidos/{id}`	Detalle con líneas	ADMIN, BARISTA, CLIENTE (propio)
11	GET	`/api/v1/pedidos`	Listado paginado por estado y fecha; un cliente solo ve los suyos	ADMIN, BARISTA, CLIENTE
12	PATCH	`/api/v1/pedidos/{id}/estado`	Cambio de estado (RN-08)	ADMIN, BARISTA
13	POST	`/api/v1/cupones`	Crear cupón	ADMIN
14	POST	`/api/v1/cupones/validar`	Validar código para un subtotal y devolver descuento	Autenticado
15	GET	`/api/v1/informes/ventas-dia?fecha=`	Nº pedidos, total, base e IVA del día	ADMIN
16	GET	`/api/v1/informes/productos-top?desde=&hasta=&limite=`	Ranking de productos más vendidos	ADMIN

5.7 Dónde aplicar cada formato de consulta

Formato	Sugerencia en CoffeeStation
Q1 Derivados	`findByCategoriaIdAndActivoTrue`, `existsByCodigoIgnoreCase`, `countByFechaHoraBetween` (turno), `findByEstadoOrderByFechaHoraAsc`, `findByNombreContainingIgnoreCase`
Q2 JPQL	Suma de ventas entregadas entre fechas; productos que no contienen un alérgeno (`JOIN` + `NOT IN`); pedidos de un cliente por email

Formato	Sugerencia en CoffeeStation
Q3 Nativa	Ranking de productos más vendidos con `LIMIT`; ventas por franja horaria con `EXTRACT(HOUR …)`
Q4 Modificación	`UPDATE Producto p SET p.disponible = :valor WHERE p.categoria.id = :id`
Q5 Proyecciones	Interfaz `ProductoCartaView` para la carta; `SELECT new …VentasDiaDto(…)` para el informe
Q6 Paginación	Endpoints 1 y 11
Q7 Specification	Filtros combinables del endpoint 1
Q8 N+1	`@EntityGraph` para cargar un pedido con sus líneas y productos (endpoint 10)

5.8 Integración

CONSUMES →

FitZone (Caso 2) · Contrato B

Al crear un pedido con `dniSocio`, llamas a `GET /api/v1/integracion/socios/{dni}/estado`. Si FitZone no responde, el pedido se crea sin descuento de socio y la respuesta lo indica.

← EXPONES PARA

CineTown (Caso 3) · Contrato A

`POST /api/v1/integracion/cupones`, que genera un cupón personal para el email del comprador de entradas.

Contratos completos en la sección 9.

9 Contratos de integración

Los cuatro contratos de esta sección son fijos: rutas, nombres de campos y códigos de respuesta deben coincidir exactamente, porque tu API la va a llamar el código de otra persona.

9.1 Reglas comunes

- Rutas bajo `/api/v1/integracion/**`, protegidas con la cabecera `X-API-KEY` (sin JWT). Clave incorrecta o ausente: 401.
- Cada aplicación arranca en su puerto: CoffeeStation `8081`, FitZone `8082`, CineTown `8083`, ParkControl `8084`. En clase, la URL del compañero será `http://<IP-del-compañero>:<puerto>`.
- URL y API key del servicio remoto se configuran en `application.yml` a partir de variables de entorno (`SAFAREYES_<SERVICIO>_URL`, `SAFAREYES_<SERVICIO>_API_KEY`).
- El cliente usa `RestClient` con tiempo de conexión y lectura de 3 segundos, y trata por separado: respuesta correcta, 4xx (rechazo de negocio), 5xx y tiempo agotado o conexión rechazada (servicio no disponible).
- La operación principal **nunca se bloquea** por un fallo de integración. La respuesta de tu endpoint incluye un bloque `integracion` que informa del resultado.
- Las API solo intercambian los datos imprescindibles (principio de minimización del RGPD): nunca nombres, teléfonos ni datos de pago.

```json
"integracion": {
  "servicio": "FITZONE",
  "estado": "OK",
  "detalle": "Socio activo: descuento del 10 % aplicado"
}
```

Estado	Significado
`OK`	La otra API respondió correctamente y se aplicó el resultado
`NO_APLICA`	No había nada que integrar (p. ej. socio sin matrícula)
`RECHAZADO`	La otra API respondió 4xx
`NO_DISPONIBLE`	5xx, tiempo agotado o sin conexión

Contrato	Expone	Consume	Momento de la llamada
A	CoffeeStation	CineTown	Tras confirmar una compra de entradas
B	FitZone	CoffeeStation	Al crear un pedido con `dniSocio`
C	ParkControl	FitZone	Tras una entrada `PERMITIDO` de un socio con matrícula
D	CineTown	ParkControl	Al pagar un ticket con `codigoCine`

9.2 Contratos

A Cupón de cine · expone CoffeeStation

`POST` `/api/v1/integracion/cupones`

PETICIÓN

JSON

```
{
  "email": "ana.lopez@correo.es",
  "origen": "CINETOWN",
  "referencia": "K7Q2M9XA"
}
```

RESPUESTA 201 CREATED

JSON

```
{
  "codigo": "CINE-4F7K2Q",
  "email": "ana.lopez@correo.es",
  "tipo": "PORCENTAJE",
  "valor": 20.00,
  "validoHasta": "2026-10-21",
  "usosMaximos": 1
}
```

- CoffeeStation decide el descuento (20 %, 7 días, 1 uso). El que llama no puede fijarlo.
- **Idempotencia:** si llega otra petición con la misma `referencia`, se devuelve el cupón ya creado con `200 OK`, sin duplicarlo.
- Errores: 400 (email no válido), 401.

B Estado de socio · expone FitZone

`GET` `/api/v1/integracion/socios/{dni}/estado`

PETICIÓN

Sin cuerpo. El DNI va en la ruta.

RESPUESTA 200 OK

JSON

```
{
  "dni": "12345678Z",
  "socioActivo": true,
  "cuotaAlCorriente": true,
  "plan": "PREMIUM"
}
```

- CoffeeStation aplica el descuento de socio solo si `socioActivo` y `cuotaAlCorriente` son `true`.
- Errores: 400 (DNI con formato no válido), 401, 404 (no es socio).